Documento de Decisiones de Diseño

Actividad Colaborativa N°3 – Proyecto Final:
Sistema de gestión de una empresa de transporte de carga

Integrantes: Jonathan Gómez Salazar y Omar Picón Meza

Curso: Programación Orientada a Objetos

Universidad de Manizales – Facultad de Ciencias e Ingenierías

Docente: Natalia Marcela Castellanos Gómez

Fecha: 6 de noviembre de 2026

1. Introducción

Una empresa de transporte de carga necesita saber, antes de enviar un camión a la carretera, si ese viaje se puede realizar; si la carga cabe en el vehículo, si sus documentos están vigentes y si el conductor tiene la licencia que exige ese tipo de vehículo. Cuando alguna de esas condiciones falla, el viaje no podrá salir. Este proyecto modela esa situación como un sistema en Python, aplicando de forma integrada los conceptos de programación orientada a objetos vistos en el módulo; clase abstracta, herencia, sobreescritura de métodos, polimorfismo, composición y excepciones personalizadas.

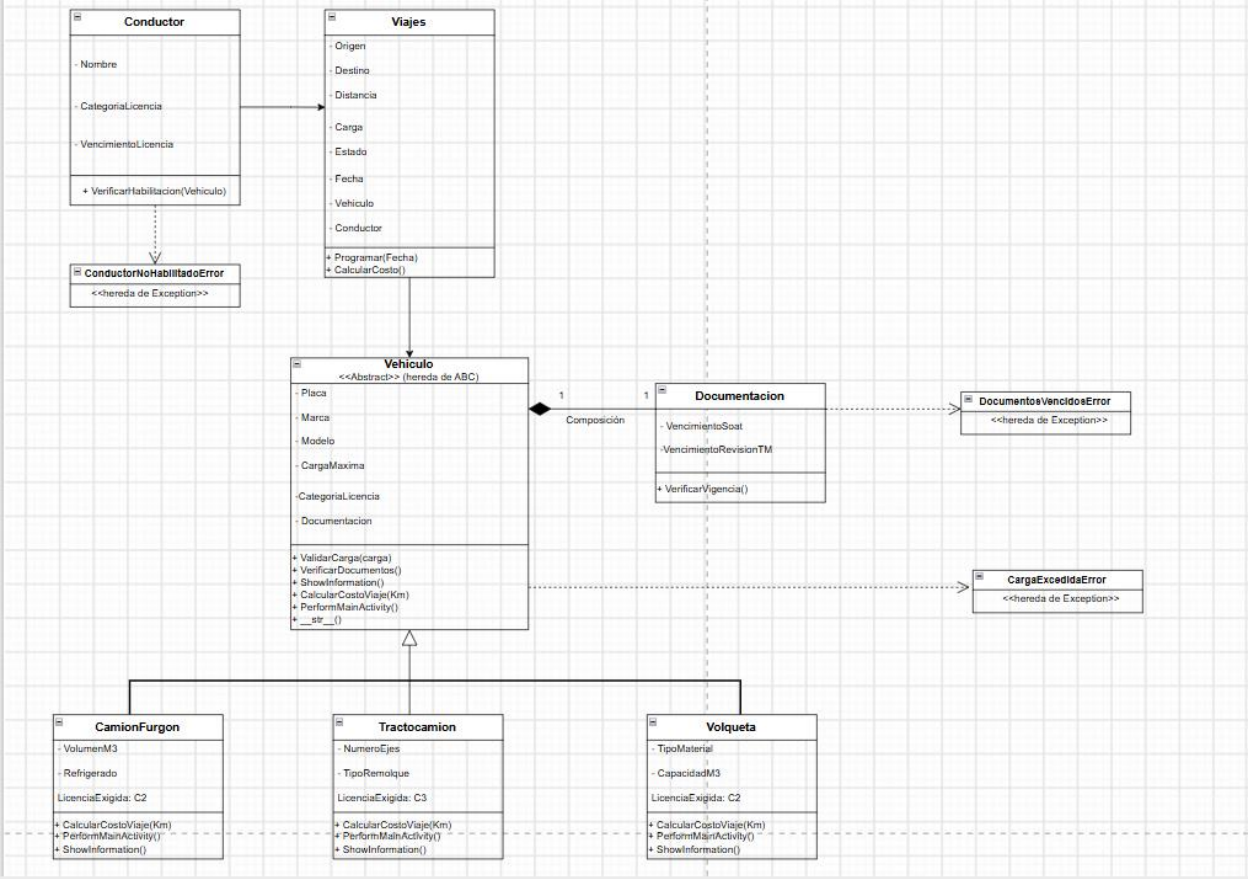
El propósito del sistema es registrar la flota de vehículos de la empresa, sus conductores y los viajes, calcular el costo de cada viaje según el tipo de vehículo y controlar las situaciones que impiden programarlo. Este documento explica y justifica las decisiones de diseño tomadas durante el desarrollo, haciendo referencia directa a las clases, atributos y métodos del proyecto, y declara el uso de herramientas de inteligencia artificial.

2. Descripción general de la solución

El proyecto se encuentra en un repositorio nuevo (transportProject), independiente del de las actividades anteriores, y se organiza en tres carpetas más el archivo principal:

- **models/:** las clases del dominio (vehiculo.py, camion_furgon.py, tractocamion.py, volqueta.py, conductor.py, documentacion.py y viaje.py).
- **exceptions/:** el archivo excepciones.py con las tres excepciones personalizadas.
- **main.py, en la raíz:** Ejecuta las pruebas con casos normales y excepcionales.

El sistema tiene siete clases: una abstracta (Vehiculo), tres hijas de ella (CamionFurgon, Tractocamion y Volqueta) y tres clases concretas independientes (Conductor, Documentacion y Viaje). Cada vehículo contiene su propia documentación (composición) y cada viaje se asocia con un vehículo y un conductor. Cuando se intenta programar un viaje se verifican, en orden, la carga, los documentos y la licencia del conductor.



3. Desarrollo de las decisiones de diseño

3.1 El problema que resuelve el sistema

El sistema resuelve un problema concreto de una empresa de carga; decidir si un viaje puede programarse y cuánto cuesta. Para eso necesita tres respuestas distintas; cuánto cobrar según el tipo de vehículo, si el vehículo y el conductor cumplen los requisitos, y qué hacer cuando no los cumplen. La primera pregunta se resuelve con herencia y polimorfismo (cada tipo de vehículo calcula su costo a su manera), la segunda con composición y asociación entre objetos, y la tercera con excepciones propias del dominio.

3.2 Clases principales y su responsabilidad

Clase	Responsabilidad
Vehiculo (abstracta)	Reúne los datos comunes de cualquier vehículo (placa, marca, modelo, carga máxima, categoría de licencia que exige y su documentación) y las verificaciones comunes.

Clase	Responsabilidad
CamionFurgon	Vehículo de furgón cerrado para mercancía empacada; puede ser refrigerado.
TractoCamion	Cabezote que arrastra un remolque para carga pesada y larga distancia.
Volqueta	Vehículo de platón basculante para material de construcción.
Documentacion	Guarda las fechas de vencimiento del SOAT y de la revisión técnico-mecánica y verifica si están vigentes.
Conductor	Guarda los datos del conductor y su licencia, y verifica si está habilitado para un vehículo.
Viaje	Relaciona un vehículo y un conductor con un origen, un destino, una distancia y una carga; programa el viaje y calcula su costo.

3.3 Vehículo como clase abstracta y la herencia

Vehículo se definió como clase abstracta (hereda de ABC) porque en una empresa de carga no existe un «vehículo genérico»; siempre es un camión furgón, un tractocamión o una volqueta. Un Vehículo suelto no sabría cuánto cobrar por kilómetro ni qué tipo de carga mueve, y por eso no debe poder instanciarse. Esta decisión se aplica con dos métodos abstractos, CalcularCostoViaje(distancia_km) y PerformMainActivity(), y se demuestra en **main.py**, donde el intento de crear un Vehículo directamente lanza un **TypeError** que se captura para que el programa continúe.

La herencia expresa una relación «es un»; un camión furgón es un vehículo, un tractocamión es un vehículo y una volqueta es un vehículo. Cada hija llama a super().__init__() y delega en Vehiculo todo lo que comparten, de modo que no repite código. Esa es la razón por la que no se hizo herencia con las otras clases; un Conductor no es un vehículo, ni un Viaje es un conductor, y forzar esas relaciones solo habría servido para cumplir el requisito.

Lo que comparten las hijas (definido en Vehiculo):

- **Atributos:** _placa, _marca, _modelo, _carga_maxima_kg, _categoria_licencia y _documentacion, con sus getters y setters validados.
- **Métodos:** ValidarCarga(), VerificarDocumentos(), ShowInformation() y __str__().

Lo propio de cada hija:

Clase	Atributos propios	Forma de calcular el costo por kilómetro
CamionFurgon	__volumen_m3, __refrigerado	3.000 pesos por km, con un recargo del 25% si es refrigerado
TractoCamion	__numero_ejes, __tipo_remolque	4.000 pesos por km más 800 por cada eje
Volqueta	__tipo_material, __capacidad_m3	3.500 pesos por km multiplicado por un factor según el material (arena 1,0; grava 1,1; escombros 1,2)

Cada hija además fija la categoría de licencia que exige su tipo de vehículo mediante una constante de clase (CATEGORIA_LICENCIA): C2 para el camión furgón y la volqueta, y C3 para el tractocamión. Es una simplificación del sistema y no una norma de tránsito exacta.

3.4 Métodos sobrescritos y polimorfismo

Las tres hijas sobrescriben tres métodos de Vehiculo, conservando el nombre, pero cambiando el resultado según sus datos propios:

- **CalcularCostoViaje(distancia_km):** para 100 km, el camión furgón refrigerado cuesta 375.000 pesos (300.000 si es seco), un tractocamión de cinco ejes cuesta 800.000 y una volqueta de escombros cuesta 420.000. El mismo método produce resultados distintos porque cada clase usa sus propios atributos (refrigeración, ejes o material).
- **PerformMainActivity():** devuelve un texto distinto según el vehículo. El furgón describe el tipo de furgón y su volumen, el tractocamión describe el tipo de remolque y la volqueta describe el material que transporta.
- **ShowInformation():** cada hija llama a super().ShowInformation() y le agrega sus atributos propios, de modo que se reutiliza lo común y solo se escribe lo que diferencia a cada clase.

El polimorfismo se evidencia en **main.py.** Se crea una única lista flota con seis vehículos de los tres tipos (dos camiones furgón, dos tractocamiones y dos volquetas) y se recorre con un solo ciclo **for** que llama a ShowInformation(), PerformMainActivity() y CalcularCostoViaje(100) sobre cada elemento, sin ningún **if** que pregunte por el tipo. Python decide en tiempo de ejecución qué versión de cada método usar. El polimorfismo también aparece en Viaje.CalcularCosto(), que le pide el costo a su vehículo sin saber de qué tipo es.

3.5 Composición entre Vehiculo y Documentacion

La composición se aplicó entre Vehiculo y Documentacion. En el constructor de Vehiculo, el vehículo crea su propio objeto Documentacion a partir de las fechas de

vencimiento del SOAT y de la revisión técnico-mecánica que recibe, en lugar de recibirlo ya construido. Los documentos pertenecen a un vehículo concreto, no se comparten con otros y no tienen sentido por separado: si el vehículo deja de existir, su documentación también.

No se resolvió con herencia porque un vehículo no es una documentación; la relación correcta es «tiene un». Si Vehiculo heredara de Documentacion, el vehículo recibiría atributos y métodos que no son suyos y se confundirían dos conceptos distintos.

3.6 Asociaciones entre Viaje, Vehiculo y Conductor

Viaje se relaciona con un Vehiculo y con un Conductor mediante asociación. A diferencia de la documentación, el vehículo y el conductor existen antes y después del viaje, y pueden participar en varios, por eso Viaje solo guarda la referencia. Los métodos SetVehiculo() y SetConductor() validan con isinstance que se reciban objetos del tipo correcto y no textos sueltos, igual que se hizo con los objetos asociados en las actividades anteriores.

3.7 Excepciones personalizadas

Tres situaciones anormales propias del dominio se controlan con excepciones propias, definidas en exceptions/excepciones.py y heredadas de Exception. Cada una recibe los datos que describen lo ocurrido y arma con ellos un mensaje claro.

Excepción	Situación que controla	Dónde se lanza
CargaExcedidaError	La carga del viaje es mayor que la que soporta el vehículo	Vehiculo.ValidarCarga()
ConductorNoHabilitadoError	La licencia del conductor está vencida o su categoría no alcanza para el vehículo	Conductor.VerificarHabilitacion()
DocumentosVencidosError	El SOAT o la revisión técnico-mecánica del	Documentacion.VerificarVigencia()

Las tres se activan a través de Viaje.Programar(fecha), que llama a las verificaciones en este orden: carga, documentos y licencia. Si alguna lanza su excepción, el método se interrumpe antes de la última línea y el viaje se queda en estado pendiente; solo cuando las tres verificaciones pasan, el estado cambia a programado. En **main.py** cada caso se captura con **try/except**, y el programa continúa hasta el final. Las validaciones de los setters (por ejemplo, una distancia negativa o una categoría de licencia inexistente) se mantuvieron como mensajes de error que no aplican el cambio, porque son validaciones de datos y no situaciones del proceso de programación.

3.8 Decisiones que cambiaron durante el desarrollo

- **Carpeta controllers/:** en el diseño inicial se había previsto una carpeta controllers/ para administrar la flota. No se creó, porque ninguna clase la necesitó y el **main.py** bastó para organizar las pruebas.
- **Categoría de licencia:** Inicialmente se pensó como un atributo que cada vehículo recibiría como parámetro. Se cambió para que cada clase hija la fije con una constante de clase, ya que la categoría depende del tipo de vehículo y no de cada vehículo individual.
- **Lista de categorías repetida:** Vehiculo y Conductor definen cada una su propia lista CATEGORIAS. Se aceptó esa pequeña repetición para que Vehiculo no dependa de la clase Conductor.
- **Contexto del sistema:** Se evaluaron dos opciones, una empresa de transporte de carga y un consultorio odontológico. Se eligió la primera porque sus tres tipos de vehículo se diferencian más en datos y cálculos propios, y eso permite justificar mejor la herencia y la sobreescritura.

4. Pruebas y verificación

Cada clase incluye un bloque de prueba rápida que se ejecuta de forma independiente con py -m models.nombre_de_la_clase, y el archivo **main.py** reúne la prueba de todo el sistema. El programa principal demuestra:

- Que no se puede crear un Vehiculo directamente.
- La creación de más de ocho objetos: siete vehículos (seis en la flota y uno con SOAT vencido), tres conductores y siete viajes, además de un objeto Documentacion por cada vehículo.
- Las validaciones de los setters, con datos inválidos que se informan sin modificar el objeto.
- El recorrido de la colección heterogénea flota con un único ciclo.
- La programación correcta de tres viajes.
- Las cuatro situaciones excepcionales: carga excedida, conductor con licencia de categoría insuficiente, conductor con licencia vencida y vehículo con el SOAT vencido.
- La continuidad del programa después de cada excepción.

Para comprobar que el código funciona y corresponde con los conceptos estudiados, el equipo ejecutó cada clase por separado y luego el **main.py** completo, comparó la salida con el resultado esperado en cada caso (por ejemplo, los costos por kilómetro de cada tipo de vehículo y el estado pendiente de los viajes que no se pueden programar) y corrigió los errores que aparecieron en el sistema.

5. Uso de herramientas de inteligencia artificial

Se utilizó Claude (Anthropic) como apoyo durante el desarrollo de la actividad.

Para qué se utilizó:

- Comprender cómo aplicar los conceptos a un contexto nuevo y comparar posibles contextos para el sistema.
- Proponer el diseño inicial de las clases, las relaciones y las excepciones.
- Generar fragmentos de código de las clases, que luego se adaptaron al estilo del proyecto.
- Revisar errores de ejecución y apoyar la redacción de este documento.

Qué propuestas se modificaron, descartaron o corrigieron:

- Se descartó la opción del consultorio odontológico y se eligió el transporte de carga.
- Se descartó la carpeta **controllers/** prevista en el diseño inicial.
- Las tarifas por kilómetro (3.000, 4.000 más 800 por eje, y 3.500 con factor por material) son valores de ejemplo propuestos por la herramienta; el equipo los revisó y los dejó como constantes de clase para poder ajustarlos fácilmente.
- Se corrigieron errores del proceso; una línea cortada al copiar **excepciones.py** que produjo un SyntaxError, y un .gitignore ubicado en la carpeta equivocada que dejó subir la carpeta __pycache__ al repositorio.

El uso de estas herramientas no sustituye la comprensión del trabajo, los dos integrantes pueden explicar cada clase, método y decisión descritos en este documento.

6. Conclusiones

El proyecto permitió aplicar en un contexto nuevo los conceptos del módulo y entender por qué cada uno resuelve un problema distinto. La clase abstracta Vehiculo garantiza que cada tipo de vehículo defina cómo calcula su costo; la herencia evita repetir los datos comunes; el polimorfismo permite recorrer la flota con un solo ciclo; la composición con Documentacion modela algo que no existe sin su dueño; y las excepciones convierten situaciones como una carga excesiva, una licencia o un SOAT vencidos en errores claros que no detienen el programa.

Las principales dificultades fueron de organización más que de lógica, como coordinar el trabajo en un mismo repositorio entre los dos integrantes, mantener el mismo estilo de código en todas las clases y resolver errores de Git (como archivos que no debían subirse). Como aprendizaje, el equipo confirmó la importancia de diseñar antes de programar, de probar cada clase antes de pasar a la siguiente y de mantener el diagrama de clases sincronizado con el código.